org.apache.commons.csv.CSVParserTest:110: $stack24 = virtualinvoke $stack23.<org.apache.commons.csv.CSVRecord: java.lang.String[] values()>()

org.apache.commons.csv.CSVParserTest:110: $stack21 = $stack6[i]

org.apache.commons.csv.CSVParserTest:110: staticinvoke <org.junit.Assert: void assertArrayEquals(java.lang.Object[],java.lang.Object[])>($stack21, $stack24) org.apache.commons.csv.CSVParserTest:109: i = 0 org.apache.commons.csv.CSVParserTest:98: $stack6[4] = $stack11 org.apache.commons.csv.CSVParserTest:98: $stack6[3] = $stack10 org.apache.commons.csv.CSVParserTest:98: $stack6[2] = $stack9 org.apache.commons.csv.CSVParserTest:98: $stack6[1] = $stack8 org.apache.commons.csv.CSVParserTest:98: $stack6[0] = $stack7 org.apache.commons.csv.CSVParserTest:98: $stack6 = newarray (java.lang.String[])[5]

org.apache.commons.csv.CSVRecord:138: return $stack1 org.apache.commons.csv.CSVParserTest:98: $stack11[2] = "abcndef" org.apache.commons.csv.CSVParserTest:98: $stack11[1] = " "world"" org.apache.commons.csv.CSVParserTest:98: $stack11[0] = ""hello"" org.apache.commons.csv.CSVParserTest:98: $stack11 = newarray (java.lang.String)[4] org.apache.commons.csv.CSVParserTest:98: $stack11[3] = "" org.apache.commons.csv.CSVParserTest:98: $stack10[0] = "" org.apache.commons.csv.CSVParserTest:98: $stack10 = newarray (java.lang.String)[1] org.apache.commons.csv.CSVParserTest:98: $stack9[2] = "" org.apache.commons.csv.CSVParserTest:98: $stack9[1] = "" org.apache.commons.csv.CSVParserTest:98: $stack9[0] = " x" org.apache.commons.csv.CSVParserTest:98: $stack9 = newarray (java.lang.String)[4] org.apache.commons.csv.CSVParserTest:98: $stack9[3] = "" org.apache.commons.csv.CSVParserTest:98: $stack8[2] = "c" org.apache.commons.csv.CSVParserTest:98: $stack8[1] = "b" org.apache.commons.csv.CSVParserTest:98: $stack8[0] = "a" org.apache.commons.csv.CSVParserTest:98: $stack8 = newarray (java.lang.String)[4] org.apache.commons.csv.CSVParserTest:98: $stack8[3] = "d" org.apache.commons.csv.CSVParserTest:98: $stack7[2] = "value3" org.apache.commons.csv.CSVParserTest:98: $stack7[1] = "value2" org.apache.commons.csv.CSVParserTest:98: $stack7[0] = "value1" org.apache.commons.csv.CSVParserTest:98: $stack7 = newarray (java.lang.String)[4] org.apache.commons.csv.CSVParserTest:98: $stack7[3] = "value4"

org.apache.commons.csv.Lexer:59: this.<org.apache.commons.csv.Lexer: org.apache.commons.csv.ExtendedBufferedReader in> = in

org.apache.commons.csv.CSVLexer:37: specialinvoke this.<org.apache.commons.csv.Lexer: void <init>(org.apache.commons.csv.CSVFormat,org.apache.commons.csv.ExtendedBufferedReader)>(format, in)

org.apache.commons.csv.ExtendedBufferedReader:40: this.<org.apache.commons.csv.ExtendedBufferedReader: int lastChar> = $i0

org.apache.commons.csv.ExtendedBufferedReader:49: specialinvoke this.<java.io.BufferedReader: void <init>(java.io.Reader)>(reader)

org.apache.commons.csv.CSVParser:122: $stack6 = new org.apache.commons.csv.ExtendedBufferedReader

org.apache.commons.csv.CSVParserTest:105: $stack12 = new org.apache.commons.csv.CSVParser

org.apache.commons.csv.CSVParser:122: specialinvoke $stack6.<org.apache.commons.csv.ExtendedBufferedReader: void <init>(java.io.Reader)>(input) org.apache.commons.csv.CSVParser:122: specialinvoke $stack5.<org.apache.commons.csv.CSVLexer: void <init>(org.apache.commons.csv.CSVFormat,org.apache.commons.csv.ExtendedBufferedReader)>(format, $stack6)

org.apache.commons.csv.ExtendedBufferedReader:-1: $i0 = (int) -2

org.apache.commons.csv.CSVParser:139: specialinvoke $stack3.<java.io.StringReader: void <init>(java.lang.String)>(input)

org.apache.commons.csv.CSVParser:139: $stack3 = new java.io.StringReader org.apache.commons.csv.CSVParser:139: specialinvoke this.<org.apache.commons.csv.CSVParser: void <init>(java.io.Reader,org.apache.commons.csv.CSVFormat)>($stack3, format)

org.apache.commons.csv.Constants:36: <org.apache.commons.csv.Constants: java.lang.Character DOUBLE_QUOTE_CHAR> = $stack0

org.apache.commons.csv.Constants:36: $stack0 = staticinvoke <java.lang.Character: java.lang.Character valueOf(char)>(34)

org.apache.commons.csv.CSVFormat:442: $stack1 = <org.apache.commons.csv.Constants: java.lang.Character DOUBLE_QUOTE_CHAR>

org.apache.commons.csv.CSVFormat:507: $stack5 = staticinvoke <org.apache.commons.csv.CSVFormat: org.apache.commons.csv.CSVFormat$CSVFormatBuilder newBuilder()>()

org.apache.commons.csv.CSVFormat:442: specialinvoke $stack0.<org.apache.commons.csv.CSVFormat$CSVFormatBuilder: void <init>(char,java.lang.Character,org.apache.commons.csv.Quote,java.lang.Character,java.lang.Character,boolean,boolean,java.lang.String,java.lang.String,java.lang.String[])>(44, $stack1, null, null, null, 0, 1, "rn", "", null)

org.apache.commons.csv.CSVFormat$CSVFormatBuilder:114: this.<org.apache.commons.csv.CSVFormat$CSVFormatBuilder: char delimiter> = delimiter org.apache.commons.csv.CSVFormat$CSVFormatBuilder:115: this.<org.apache.commons.csv.CSVFormat$CSVFormatBuilder: java.lang.Character quoteChar> = quoteChar org.apache.commons.csv.CSVFormat$CSVFormatBuilder:118: this.<org.apache.commons.csv.CSVFormat$CSVFormatBuilder: java.lang.Character escape> = escape

org.apache.commons.csv.CSVFormat$CSVFormatBuilder:148: $stack11 = this.<org.apache.commons.csv.CSVFormat$CSVFormatBuilder: char delimiter>

org.apache.commons.csv.CSVFormat:507: $stack7 = virtualinvoke $stack6.<org.apache.commons.csv.CSVFormat$CSVFormatBuilder: org.apache.commons.csv.CSVFormat build()>()

org.apache.commons.csv.CSVFormat$CSVFormatBuilder:148: specialinvoke $stack1.<org.apache.commons.csv.CSVFormat: void <init>(char,java.lang.Character,org.apache.commons.csv.Quote,java.lang.Character,java.lang.Character,boolean,boolean,java.lang.String,java.lang.String,java.lang.String[])>($stack11, $stack10, $stack9, $stack8, $stack7, $stack6, $stack5, $stack4, $stack3, $stack2) org.apache.commons.csv.CSVFormat$CSVFormatBuilder:148: $stack10 = this.<org.apache.commons.csv.CSVFormat$CSVFormatBuilder: java.lang.Character quoteChar> org.apache.commons.csv.CSVFormat$CSVFormatBuilder:148: $stack7 = this.<org.apache.commons.csv.CSVFormat$CSVFormatBuilder: java.lang.Character escape>

org.apache.commons.csv.CSVFormat:606: this.<org.apache.commons.csv.CSVFormat: char delimiter> = delimiter org.apache.commons.csv.CSVFormat:607: this.<org.apache.commons.csv.CSVFormat: java.lang.Character quoteChar> = quoteChar org.apache.commons.csv.CSVFormat:610: this.<org.apache.commons.csv.CSVFormat: java.lang.Character escape> = escape

org.apache.commons.csv.CSVFormat:710: $stack1 = this.<org.apache.commons.csv.CSVFormat: char delimiter>

org.apache.commons.csv.Lexer:60: $stack3 = virtualinvoke format.<org.apache.commons.csv.CSVFormat: char getDelimiter()>()

org.apache.commons.csv.CSVFormat:761: $stack1 = this.<org.apache.commons.csv.CSVFormat: java.lang.Character quoteChar>

org.apache.commons.csv.Lexer:62: $stack6 = virtualinvoke format.<org.apache.commons.csv.CSVFormat: java.lang.Character getQuoteChar()>()

org.apache.commons.csv.CSVFormat:719: $stack1 = this.<org.apache.commons.csv.CSVFormat: java.lang.Character escape>

org.apache.commons.csv.Lexer:61: $stack4 = virtualinvoke format.<org.apache.commons.csv.CSVFormat: java.lang.Character getEscape()>()

org.apache.commons.csv.CSVParserTest:105: specialinvoke $stack12.<org.apache.commons.csv.CSVParser: void <init>(java.lang.String,org.apache.commons.csv.CSVFormat)>("value1,value2,value3,value4rna,b,c,drn x,,,rnrn"""hello"""," ""world""","abcndef",rn", $stack13)

org.apache.commons.csv.CSVParser:93: specialinvoke $stack4.<org.apache.commons.csv.Token: void <init>()>()

org.apache.commons.csv.CSVParser:92: $stack3 = new java.util.ArrayList

org.apache.commons.csv.CSVParser:92: specialinvoke $stack3.<java.util.ArrayList: void <init>()>()

org.apache.commons.csv.CSVParser:92: this.<org.apache.commons.csv.CSVParser: java.util.List record> = $stack3

org.apache.commons.csv.CSVParser:185: $stack4 = this.<org.apache.commons.csv.CSVParser: java.util.List record>

org.apache.commons.csv.CSVParser:195: interfaceinvoke $stack68.<java.util.List: boolean add(java.lang.Object)>($stack69)

org.apache.commons.csv.CSVParser:237: $stack4 = virtualinvoke this.<org.apache.commons.csv.CSVParser: org.apache.commons.csv.CSVRecord nextRecord()>()

org.apache.commons.csv.CSVParser:190: tableswitch($stack13) { case 1: org.apache.commons.csv.CSVParser:195: $stack69 = virtualinvoke $stack67.<java.lang.StringBuilder: java.lang.String toString()>()

org.apache.commons.csv.CSVParser:195: $stack68 = this.<org.apache.commons.csv.CSVParser: java.util.List record>

org.apache.commons.csv.CSVParserTest:106: $stack14 = virtualinvoke $stack12.<org.apache.commons.csv.CSVParser: java.util.List getRecords()>()

org.apache.commons.csv.Token:55: this.<org.apache.commons.csv.Token: java.lang.StringBuilder content> = $stack2

org.apache.commons.csv.Token:55: specialinvoke $stack2.<java.lang.StringBuilder: void <init>(int)>(50)

org.apache.commons.csv.Token:55: $stack2 = new java.lang.StringBuilder

org.apache.commons.csv.Token$Type:34: specialinvoke this.<java.lang.Enum: void <init>(java.lang.String,int)>(l1, l2)

org.apache.commons.csv.Token$Type:48: $stack4 = new org.apache.commons.csv.Token$Type org.apache.commons.csv.Token$Type:42: $stack2 = new org.apache.commons.csv.Token$Type org.apache.commons.csv.Token$Type:45: $stack3 = new org.apache.commons.csv.Token$Type org.apache.commons.csv.Token$Type:39: $stack1 = new org.apache.commons.csv.Token$Type org.apache.commons.csv.Token$Type:36: $stack0 = new org.apache.commons.csv.Token$Type org.apache.commons.csv.Token$Type:39: specialinvoke $stack1.<org.apache.commons.csv.Token$Type: void <init>(java.lang.String,int)>("TOKEN", 1) org.apache.commons.csv.Token$Type:36: specialinvoke $stack0.<org.apache.commons.csv.Token$Type: void <init>(java.lang.String,int)>("INVALID", 0) org.apache.commons.csv.Token$Type:48: specialinvoke $stack4.<org.apache.commons.csv.Token$Type: void <init>(java.lang.String,int)>("COMMENT", 4) org.apache.commons.csv.Token$Type:45: specialinvoke $stack3.<org.apache.commons.csv.Token$Type: void <init>(java.lang.String,int)>("EORECORD", 3) org.apache.commons.csv.Token$Type:42: specialinvoke $stack2.<org.apache.commons.csv.Token$Type: void <init>(java.lang.String,int)>("EOF", 2)

org.apache.commons.csv.Token$Type:36: <org.apache.commons.csv.Token$Type: org.apache.commons.csv.Token$Type INVALID> = $stack0

org.apache.commons.csv.Token$Type:34: return

org.apache.commons.csv.CSVLexer:96: $stack16 = <org.apache.commons.csv.Token$Type: org.apache.commons.csv.Token$Type INVALID>

org.apache.commons.csv.CSVParser:189: virtualinvoke $stack7.<org.apache.commons.csv.Lexer: org.apache.commons.csv.Token nextToken(org.apache.commons.csv.Token)>($stack6)

org.apache.commons.csv.Token:62: $stack2 = <org.apache.commons.csv.Token$Type: org.apache.commons.csv.Token$Type INVALID>

org.apache.commons.csv.CSVParser:188: virtualinvoke $stack5.<org.apache.commons.csv.Token: void reset()>()

org.apache.commons.csv.Token$Type:34: $stack6 = <org.apache.commons.csv.Token$Type: org.apache.commons.csv.Token$Type INVALID>

org.apache.commons.csv.Token$Type:39: <org.apache.commons.csv.Token$Type: org.apache.commons.csv.Token$Type TOKEN> = $stack1

org.apache.commons.csv.CSVLexer:160: $stack29 = <org.apache.commons.csv.Token$Type: org.apache.commons.csv.Token$Type TOKEN>

org.apache.commons.csv.CSVLexer:124: specialinvoke this.<org.apache.commons.csv.CSVLexer: org.apache.commons.csv.Token parseSimpleToken(org.apache.commons.csv.Token,int)>(token, c)

org.apache.commons.csv.CSVLexer:116: if $stack21 == 0

org.apache.commons.csv.CSVLexer:108: $stack27 = <org.apache.commons.csv.Token$Type: org.apache.commons.csv.Token$Type TOKEN> org.apache.commons.csv.Token$Type:34: $stack7 = <org.apache.commons.csv.Token$Type: org.apache.commons.csv.Token$Type TOKEN>

org.apache.commons.csv.Token$Type:42: <org.apache.commons.csv.Token$Type: org.apache.commons.csv.Token$Type EOF> = $stack2

org.apache.commons.csv.Token$Type:34: $stack8 = <org.apache.commons.csv.Token$Type: org.apache.commons.csv.Token$Type EOF>

org.apache.commons.csv.Token$Type:45: <org.apache.commons.csv.Token$Type: org.apache.commons.csv.Token$Type EORECORD> = $stack3

org.apache.commons.csv.CSVLexer:153: $stack35 = <org.apache.commons.csv.Token$Type: org.apache.commons.csv.Token$Type EORECORD>

org.apache.commons.csv.CSVLexer:112: $stack26 = <org.apache.commons.csv.Token$Type: org.apache.commons.csv.Token$Type EORECORD> org.apache.commons.csv.Token$Type:34: $stack9 = <org.apache.commons.csv.Token$Type: org.apache.commons.csv.Token$Type EORECORD>

org.apache.commons.csv.Token$Type:48: <org.apache.commons.csv.Token$Type: org.apache.commons.csv.Token$Type COMMENT> = $stack4

org.apache.commons.csv.Token$Type:34: $stack10 = <org.apache.commons.csv.Token$Type: org.apache.commons.csv.Token$Type COMMENT>

org.apache.commons.csv.Token$Type:34: <org.apache.commons.csv.Token$Type: org.apache.commons.csv.Token$Type[] $VALUES> = $stack5

org.apache.commons.csv.Token$Type:34: $stack5 = newarray (org.apache.commons.csv.Token$Type)[5] org.apache.commons.csv.Token$Type:34: $stack5[4] = $stack10 org.apache.commons.csv.Token$Type:34: $stack5[2] = $stack8 org.apache.commons.csv.Token$Type:34: $stack5[3] = $stack9 org.apache.commons.csv.Token$Type:34: $stack5[1] = $stack7 org.apache.commons.csv.Token$Type:34: $stack5[0] = $stack6

org.apache.commons.csv.Token$Type:34: $stack0 = <org.apache.commons.csv.Token$Type: org.apache.commons.csv.Token$Type[] $VALUES>

org.apache.commons.csv.CSVParser$2:190: $stack1 = staticinvoke <org.apache.commons.csv.Token$Type: org.apache.commons.csv.Token$Type[] values()>()

org.apache.commons.csv.Token:61: $stack1 = this.<org.apache.commons.csv.Token: java.lang.StringBuilder content>

org.apache.commons.csv.Token:61: virtualinvoke $stack1.<java.lang.StringBuilder: void setLength(int)>(0)

org.apache.commons.csv.CSVLexer:171: virtualinvoke $stack9.<java.lang.StringBuilder: java.lang.StringBuilder append(char)>($stack8)

org.apache.commons.csv.CSVLexer:171: $stack9 = tkn.<org.apache.commons.csv.Token: java.lang.StringBuilder content>

org.apache.commons.csv.CSVLexer:171: $stack8 = (char) c

org.apache.commons.csv.CSVLexer:162: if $stack7 == 0

org.apache.commons.csv.ExtendedBufferedReader:50: return

org.apache.commons.csv.ExtendedBufferedReader:71: $stack1 = this.<org.apache.commons.csv.ExtendedBufferedReader: int lastChar>

org.apache.commons.csv.CSVLexer:55: lastChar = virtualinvoke $stack6.<org.apache.commons.csv.ExtendedBufferedReader: int getLastChar()>() org.apache.commons.csv.ExtendedBufferedReader:58: this.<org.apache.commons.csv.ExtendedBufferedReader: int lastChar> = $stack2

org.apache.commons.csv.CSVLexer:58: $stack9 = virtualinvoke $stack8.<org.apache.commons.csv.ExtendedBufferedReader: int read()>()

org.apache.commons.csv.ExtendedBufferedReader:54: $stack2 = specialinvoke this.<java.io.BufferedReader: int read()>()

org.apache.commons.csv.Lexer:135: c = virtualinvoke $stack7.<org.apache.commons.csv.ExtendedBufferedReader: int read()>() org.apache.commons.csv.CSVLexer:172: c = virtualinvoke $stack11.<org.apache.commons.csv.ExtendedBufferedReader: int read()>()

org.apache.commons.csv.Lexer:62: this.<org.apache.commons.csv.Lexer: char quoteChar> = $stack7

org.apache.commons.csv.Lexer:69: return $stack2

org.apache.commons.csv.Lexer:62: $stack7 = specialinvoke this.<org.apache.commons.csv.Lexer: char mapNullToDisabled(java.lang.Character)>($stack6) org.apache.commons.csv.Lexer:61: $stack5 = specialinvoke this.<org.apache.commons.csv.Lexer: char mapNullToDisabled(java.lang.Character)>($stack4)

org.apache.commons.csv.Lexer:69: $stack2 = virtualinvoke c.<java.lang.Character: char charValue()>() org.apache.commons.csv.Lexer:69: $stack2 = 65534

org.apache.commons.csv.Lexer:61: this.<org.apache.commons.csv.Lexer: char escape> = $stack5

org.apache.commons.csv.Lexer:60: this.<org.apache.commons.csv.Lexer: char delimiter> = $stack3

org.apache.commons.csv.Lexer:135: $stack7 = this.<org.apache.commons.csv.Lexer: org.apache.commons.csv.ExtendedBufferedReader in> org.apache.commons.csv.CSVFormat:710: return $stack1

org.apache.commons.csv.CSVLexer:63: eol = virtualinvoke this.<org.apache.commons.csv.CSVLexer: boolean readEndOfLine(int)>($stack9) org.apache.commons.csv.CSVLexer:152: $stack4 = virtualinvoke this.<org.apache.commons.csv.CSVLexer: boolean readEndOfLine(int)>(c)

org.apache.commons.csv.CSVLexer:58: $stack8 = this.<org.apache.commons.csv.CSVLexer: org.apache.commons.csv.ExtendedBufferedReader in> org.apache.commons.csv.CSVLexer:172: $stack11 = this.<org.apache.commons.csv.CSVLexer: org.apache.commons.csv.ExtendedBufferedReader in>

org.apache.commons.csv.Lexer:167: $stack2 = this.<org.apache.commons.csv.Lexer: char delimiter>

org.apache.commons.csv.CSVLexer:106: $stack19 = virtualinvoke this.<org.apache.commons.csv.CSVLexer: boolean isDelimiter(int)>(c) org.apache.commons.csv.CSVLexer:159: $stack6 = virtualinvoke this.<org.apache.commons.csv.CSVLexer: boolean isDelimiter(int)>(c)

org.apache.commons.csv.CSVFormat:719: return $stack1

org.apache.commons.csv.Lexer:69: if c != null

org.apache.commons.csv.CSVFormat:761: return $stack1

org.apache.commons.csv.Lexer:171: $stack2 = this.<org.apache.commons.csv.Lexer: char escape>

org.apache.commons.csv.CSVLexer:162: $stack7 = virtualinvoke this.<org.apache.commons.csv.CSVLexer: boolean isEscape(int)>(c)

org.apache.commons.csv.Lexer:175: $stack2 = this.<org.apache.commons.csv.Lexer: char quoteChar>

org.apache.commons.csv.CSVLexer:113: $stack20 = virtualinvoke this.<org.apache.commons.csv.CSVLexer: boolean isQuoteChar(int)>(c)

org.apache.commons.csv.CSVParser:195: $stack67 = $stack66.<org.apache.commons.csv.Token: java.lang.StringBuilder content>

org.apache.commons.csv.CSVParser:190: $stack13 = $stack11[$stack12]

org.apache.commons.csv.CSVParser:190: $stack11 = <org.apache.commons.csv.CSVParser$2: int[] $SwitchMap$org$apache$commonscsvToken$Type>

org.apache.commons.csv.CSVParser:190: $stack12 = virtualinvoke $stack10.<org.apache.commons.csv.Token$Type: int ordinal()>()

org.apache.commons.csv.CSVParser$2:190: <org.apache.commons.csv.CSVParser$2: int[] $SwitchMap$org$apache$commonscsvToken$Type> = $stack3

org.apache.commons.csv.CSVParser$2:190: $stack3 = newarray (int)[$stack2]

org.apache.commons.csv.CSVParser:192: $stack74 = virtualinvoke $stack72.<java.lang.StringBuilder: java.lang.String toString()>()

org.apache.commons.csv.CSVParser:192: $stack72 = $stack71.<org.apache.commons.csv.Token: java.lang.StringBuilder content>

org.apache.commons.csv.CSVParser:219: $stack33 = this.<org.apache.commons.csv.CSVParser: java.util.List record>

org.apache.commons.csv.CSVParser:190: $stack10 = $stack9.<org.apache.commons.csv.Token: org.apache.commons.csv.Token$Type type>

org.apache.commons.csv.CSVParser:192: interfaceinvoke $stack73.<java.util.List: boolean add(java.lang.Object)>($stack74)

org.apache.commons.csv.CSVParser:192: $stack73 = this.<org.apache.commons.csv.CSVParser: java.util.List record>

org.apache.commons.csv.CSVParser:185: interfaceinvoke $stack4.<java.util.List: void clear()>()

org.apache.commons.csv.CSVParser:216: if $stack28 != 0

org.apache.commons.csv.CSVParser:216: $stack28 = interfaceinvoke $stack27.<java.util.List: boolean isEmpty()>()

org.apache.commons.csv.CSVParser:216: $stack27 = this.<org.apache.commons.csv.CSVParser: java.util.List record>

org.apache.commons.csv.CSVLexer:112: token.<org.apache.commons.csv.Token: org.apache.commons.csv.Token$Type type> = $stack26

org.apache.commons.csv.CSVLexer:108: token.<org.apache.commons.csv.Token: org.apache.commons.csv.Token$Type type> = $stack27

org.apache.commons.csv.CSVLexer:153: tkn.<org.apache.commons.csv.Token: org.apache.commons.csv.Token$Type type> = $stack35

org.apache.commons.csv.CSVLexer:160: tkn.<org.apache.commons.csv.Token: org.apache.commons.csv.Token$Type type> = $stack29

org.apache.commons.csv.CSVLexer:109: if eol == 0

org.apache.commons.csv.CSVLexer:106: if $stack19 == 0

org.apache.commons.csv.CSVLexer:152: if $stack4 == 0

org.apache.commons.csv.CSVLexer:159: if $stack6 == 0

org.apache.commons.csv.CSVParser:219: $stack36 = newarray (java.lang.String)[$stack35]

org.apache.commons.csv.CSVParser:219: $stack35 = interfaceinvoke $stack33.<java.util.List: int size()>()

org.apache.commons.csv.CSVParser:219: $stack34 = this.<org.apache.commons.csv.CSVParser: java.util.List record>

org.apache.commons.csv.CSVParser:219: $stack37 = interfaceinvoke $stack34.<java.util.List: java.lang.Object[] toArray(java.lang.Object[])>($stack36)

org.apache.commons.csv.CSVParser:219: $stack40 = (java.lang.String[]) $stack37

org.apache.commons.csv.Token:62: this.<org.apache.commons.csv.Token: org.apache.commons.csv.Token$Type type> = $stack2

org.apache.commons.csv.CSVLexer:96: $stack17 = token.<org.apache.commons.csv.Token: org.apache.commons.csv.Token$Type type>

org.apache.commons.csv.CSVLexer:116: $stack21 = virtualinvoke this.<org.apache.commons.csv.CSVLexer: boolean isEndOfFile(int)>(c)

org.apache.commons.csv.CSVLexer:82: if $stack43 == 0

org.apache.commons.csv.Lexer:163: return $stack2

org.apache.commons.csv.CSVLexer:82: $stack43 = virtualinvoke this.<org.apache.commons.csv.CSVLexer: boolean isEndOfFile(int)>(c) org.apache.commons.csv.CSVLexer:155: $stack5 = virtualinvoke this.<org.apache.commons.csv.CSVLexer: boolean isEndOfFile(int)>(c)

org.apache.commons.csv.Lexer:163: $stack2 = 0

org.apache.commons.csv.CSVLexer:96: if $stack17 != $stack16

org.apache.commons.csv.CSVLexer:88: if $stack15 == 0

org.apache.commons.csv.Lexer:156: return $stack2

org.apache.commons.csv.CSVLexer:88: $stack15 = virtualinvoke this.<org.apache.commons.csv.CSVLexer: boolean isStartOfLine(int)>(lastChar)

org.apache.commons.csv.CSVLexer:58: c = $stack9

org.apache.commons.csv.ExtendedBufferedReader:59: return $stack5

org.apache.commons.csv.ExtendedBufferedReader:59: $stack5 = this.<org.apache.commons.csv.ExtendedBufferedReader: int lastChar>

org.apache.commons.csv.ExtendedBufferedReader:71: return $stack1

org.apache.commons.csv.Lexer:156: if c == 10

org.apache.commons.csv.Lexer:137: if c != 13

org.apache.commons.csv.Lexer:137: if c == 10

org.apache.commons.csv.Lexer:137: return $stack2

org.apache.commons.csv.Lexer:137: $stack2 = 1 org.apache.commons.csv.Lexer:137: $stack2 = 0

org.apache.commons.csv.Lexer:167: return $stack3

org.apache.commons.csv.Lexer:163: if c != $i0

org.apache.commons.csv.Lexer:-1: $i0 = (int) -1

org.apache.commons.csv.Lexer:156: $stack2 = 0

org.apache.commons.csv.Lexer:156: $stack2 = 1

org.apache.commons.csv.Lexer:167: $stack3 = 0 org.apache.commons.csv.Lexer:167: $stack3 = 1

org.apache.commons.csv.Lexer:167: if c != $i0

org.apache.commons.csv.Lexer:-1: $i0 = (int) $stack2

org.apache.commons.csv.CSVLexer:113: if $stack20 == 0

org.apache.commons.csv.Lexer:175: return $stack3

org.apache.commons.csv.Lexer:175: $stack3 = 0

org.apache.commons.csv.Lexer:175: if c != $i0

org.apache.commons.csv.Lexer:171: return $stack3

org.apache.commons.csv.CSVLexer:155: if $stack5 == 0

org.apache.commons.csv.Lexer:171: $stack3 = 0

org.apache.commons.csv.Lexer:171: if c != $i0

org.apache.commons.csv.Token$Type:34: return $stack2

org.apache.commons.csv.Token$Type:34: $stack2 = (org.apache.commons.csv.Token$Type[]) $stack1

org.apache.commons.csv.Token$Type:34: $stack1 = virtualinvoke $stack0.<java.lang.Object: java.lang.Object clone()>()

org.apache.commons.csv.CSVParser$2:190: $stack2 = lengthof $stack1

org.apache.commons.csv.CSVParser:219: specialinvoke $stack32.<org.apache.commons.csv.CSVRecord: void <init>(java.lang.String[],java.util.Map,java.lang.String,long)>($stack40, $stack39, $stack31, $stack38)

org.apache.commons.csv.CSVRecord:51: $stack6 = values

org.apache.commons.csv.CSVRecord:51: if values == null

org.apache.commons.csv.CSVRecord:51: this.<org.apache.commons.csv.CSVRecord: java.lang.String[] values> = $stack6

org.apache.commons.csv.CSVRecord:138: $stack1 = this.<org.apache.commons.csv.CSVRecord: java.lang.String[] values>

org.apache.commons.csv.CSVParserTest:110: $stack23 = (org.apache.commons.csv.CSVRecord) $stack22 org.apache.commons.csv.CSVParserTest:110: $stack22 = interfaceinvoke $stack14.<java.util.List: java.lang.Object get(int)>(i)